



ADDING CARTS TO YOUR EXPERIENCE

TABLE OF CONTENTS

Introduction.....	2
Carts.....	2
API Quick Reference.....	5
Troubleshooting.....	5
Contacting the Team	5
Document Change Log	6
Next Steps.....	6

Adding Carts to Your Experience

Last Updated: 01/30/2024

Read this guide to learn how to add Carts to your experience.

TIPS:

- Before using this guide, read [Using Nike APIs](#) and [Cart & Checkout Overview](#).
- Use this guide as a supplement to the API Reference for detailed use cases. See [API Quick Reference](#) for links to all the API Reference docs discussed here.

Introduction

What is a Cart?

In e-commerce, the shopping cart (also called a bag) allows consumers to collect and compare products that they are considering for purchase, but without requiring membership or entering any shipping and billing information.

At Nike, a cart contains the following:

- Products and services with respective prices, discounts, and quantities
- Promotion codes
- Totals

See also [How Is a Wish List different From a Cart?](#) and [What Is a Checkout?](#).

Carts

Manage a consumer's shopping cart and get product pricing

Check if a product can be purchased or not

Now that you know what a cart is, let's explore how to add it to your experience.

Step 1: Create the Cart

The first step in managing a consumer's cart is to create the cart using the [Carts API](#). For example, this could be done when the consumer chooses to add their first product to the cart.

To create the cart, execute a request to the [Create or Update a Cart by Cart ID](#) or [Create or Update a Cart by Filter Criteria](#) endpoint.

TIP: A cart is owned by one consumer (member, guest, or employee) who must be authenticated. If an attempt is made to manage a cart when no, or incorrect, authentication is provided, the Carts API returns an error response. See [Authorization](#) for more information.

Sample [Create or Update a Cart by Cart ID](#) PUT request URI:

```
https://api.nike.com/buy/carts/v2/61bc115b-16e5-43b5-bcaf-dd6168c543f8
```

When you successfully create the cart, the response contains product pricing. If the consumer is a member who has saved a shipping address, the response contains their recipient (contact) information and default shipping address from their Nike profile.

Step 2: Get a Cart

Now that the cart is created, you can display the cart to the consumer by calling the Cart Views API. This allows the consumer to continue shopping and view the cart details again later.

NOTE: The Cart Views API operates asynchronously. This means that after you execute the initial request, you call another endpoint to get the result. See [Using Nike APIs](#) for more details.

Step 2a: Send a PUT to Cart Views

Execute a PUT request to [Request Cart Views](#) to initiate a job to generate a view of a cart. You will need to generate and send a unique `jobId` for each request.

Sample PUT request URI:

```
https://api.nike.com/buy/cart_views/v1/{jobId}
```

The request for cart views can be made in one of two ways:

1. Request by Country, Brand, and Channel

Sample Request Body:

```
{
  "request": {
    "cart": {
      "country": "US",
      "brand": "NIKE",
      "channel": "NIKECOM"
    },
    "locale": "en_US"
  }
}
```

2. Request by Cart ID

Sample Request Body:

```
{
  "request": {
    "cart": {
      "id": "9af84d4b-6a1f-4899-af57-b4379f966913"
    },
    "locale": "en_US",
    "fulfillmentDetails": {
      "type": "SHIP",
      "location": {
        "type": "address/shipping",
        "postalAddress": {
          "postalCode": "97005",
          "country": "US"
        }
      }
    }
  }
}
```

TIPS:

- All Carts Views PUT requests must include a `locale`, e.g. "en_US"
- See the [API Reference](#) for the latest endpoint details

OPTIONAL: Show Total Shipping Price in a Cart Summary UX

If you are showing a Cart Summary UX, you may want to show a total shipping price separately from the total product price for the cart. Here the total shipping amount is a placeholder, used only for showing the consumer an estimated total order amount before they enter the checkout process.

To do this, optionally include some of the consumer's shipping details (e.g. `postalCode` and `country`) in a `fulfillmentDetails` object in the request.

Sample object:

```
"fulfillmentDetails": {
  "type": "SHIP",
  "location": {
    "type": "address/shipping",
    "postalAddress": {
      "postalCode": "97005",
      "country": "US"
    }
  }
}
```

Send the ship location details to get the cheapest shipping offering in the response. Or, if known, pass in the fulfillment details from the cart (e.g., shipping address, email info, or store number) and the cheapest from a wider array of fulfillment offerings will be returned in the response.

In any case, the total shipping price is returned in the response in totals. `fulfillment`:

```
"totals": {
  "items": {
    "total": 85,
    "details": {
      "price": 85,
      "discount": 8.5
    }
  },
  "valueAddedServices": {
    "total": 0,
    "details": {
      "price": 0,
      "discount": 0
    }
  },
  "fulfillment": {
    "total": 0,
    "details": {
      "price": 5,
      "discount": 5
    }
  },
  "taxes": {
    "total": 8.5,
    "details": {
      "items": {
        "tax": 8.5,
        "type": "SALESTAX"
      },
      "valueAddedServices": {
        "tax": 0,
        "type": "SALESTAX"
      }
    }
  },
  "total": 85
},
```

The fulfillment type (e.g. “SHIP”) is returned in response.items. `fulfillmentDetails`:

```
"fulfillmentDetails": {
  "type": "SHIP"
},
```

NOTES:

- Fulfillment offerings are supported for carts containing digital gift cards
- Fulfillment Offerings are only added for countries on the global platform, e.g., US, CN, and Western Europe.
[Contact the team](#) for additional info.

Step 2b: Retrieve the Cart Views Job

Execute a GET request to the [Retrieve Cart Views Job](#) endpoint to get the actual cart details. The job expires in 60 seconds.

Step 3: Modify the Cart

To add or remove products, services, and promotion codes from a cart, execute a request to the [Modify a Cart by Cart ID](#) or [Modify a Cart by Filter Criteria](#) endpoint.

TIP: Prices and subtotals are recalculated and returned in the response to each request.

To delete **all** the products in the cart, execute a request to [Delete All Items from a Cart by Cart ID](#) or [Delete All Items from a Cart by Filter Criteria](#) endpoints.

The delete operation is optional, even if the cart is empty; member carts will automatically purge from storage after 90 days of inactivity, while guest carts will purge after 30 days.

TIP: For more info on how to use the `?filter` query parameter, see [Using Nike APIs](#).

API Quick Reference

Carts V2

- [Create or Update a Cart by Cart ID](#)
- [Modify a Cart by Cart ID](#)
- [Delete All Items from a Cart by Cart ID](#)
- [Create or Update a Cart by Filter Criteria](#)
- [Modify a Cart by Filter Criteria](#)
- [Delete All Items from a Cart by Filter Criteria](#)

Cart Views

- [Request Cart Views](#)
- [Retrieve Cart Views Job](#)

Troubleshooting

Listed below are ways to troubleshoot unexpected responses using this API.

Use Troubleshooting Tools

- Use the general troubleshooting tips in the [Using Nike APIs](#) guide.
- Use a Splunk query (requires access) to check for issues with your request.
- Contact the Buy team on the [#cic-order-integration](#) Slack channel for assistance.

Contacting the Team

Need to contact the Cart & Checkout team?

Slack	#cic-order-integration
Confluence Space	CiC Order Capture
Team Contacts	Lalah Sencar

Document Change Log

Summary	Date
Converted to stand alone guide, formerly in Cart & Checkout guide	04/23/2020
Changed cart 'get' operations to use Cart Views instead of Carts v2, moved Cart Reviews content to Cart & Checkout guide	05/09/2022
Add discussion of Fulfillment Details with Cart Views	07/20/2022
Updated team contacts	01/30/2024

Next Steps

You've learned how to add Carts to your experience. Here are some related topics.

- [Wishlist](#)
- [Checkout](#)
- [Fulfillment Offerings](#)
- [Payment](#)
- [Using Nike APIs](#)
- [Glossary](#)
- [Supported Countries](#)